PROGRAMMING LAB: ALGORITHMS FOR EXPERIMENTS 1–15

EXPERIMENT 1 – LEXICAL ANALYZER IMPLEMENTATION USING A C PROGRAM

1. Start.
2. Open input.txt in read mode.
3. Initialize line_no = 1, num = 0 and i = 0.
4. Read characters from the file using getc() until EOF.
5. If a character is a digit, initialize num and continue reading digits to form the complete number.
6. Convert the digits to their numeric value and display the value as a number.
7. If a character is an alphabetic letter, store it in a string and continue reading letters or digits to form a word.
8. Compare the word with the keywords: for, while, do, switch, case, struct, int, float, double, char, if, else, return, void and main.
9. If the word matches a keyword, display it as a keyword; otherwise, display it as an identifier.
10. For a relational-operator character, inspect the next character and identify the operator (for example: =, ==, !=, <, <=, > or >=).
11. If the character is a newline, increment the line number.
12. Otherwise, display the character as a special symbol. Ignore spaces, tabs and comments as specified in the experiment aim.
13. Continue until EOF, then stop.

EXPERIMENT 2 – LEXICAL ANALYZER USING LEX

1. Start.
2. Define patterns for delimiters, whitespace, letters, digits, identifiers and numbers.
3. Define patterns for the keywords if, else and then.
4. Define patterns for relational operators (as listed in the notes: <, <=, >, >=, == and <>).
5. Read the input using yylex().
6. If the input matches whitespace, perform no action.
7. If it matches if, else or then, display it as a keyword.
8. If it matches the number pattern, display it as a number.
9. If it matches a relational-operator pattern, display the corresponding relational operator.
10. If it matches the identifier pattern, display it as an identifier.
11. Continue matching until the entire input has been processed.
12. Stop.

EXPERIMENT 3 – COUNT LINES, SPACES, TABS AND OTHER CHARACTERS
==========

1. Start.
2. Initialize four counters to zero: line_count (lc), space_count (sc), tab_count (tc) and other_char_count (ch).
3. Read the input text character by character using the Lex analyzer.
4. Check the type of each character:
   a. If it is a newline, increment lc.
   b. If it is a space, increment sc.
   c. If it is a tab, increment tc.
   d. Otherwise, increment ch.
5. Continue reading and classifying characters until the end of input (the notes use Ctrl+D to terminate input).
6. Display the number of lines, spaces, tabs and other characters.
7. Stop.

==========
EXPERIMENT 4 – REPLACE EVERY SUBSTRING "abc" WITH "ABC"
==========

1. Start the program.
2. Define the required header files and variables.
3. Read the input string containing the substring abc.
4. Find the length of the input string.
5. Initialize i = 0 and scan the string from the beginning up to the third-last character.
6. At each position, check whether the three consecutive characters are 'a', 'b' and 'c'.
7. If all three conditions are true, the substring abc has been found.
8. Replace the three characters with 'A', 'B' and 'C', respectively.
9. Continue scanning the remaining characters to find any other occurrence of abc.
10. After scanning the complete string, print the modified string.
11. Stop.

==========
EXPERIMENT 5 – COUNT VOWELS AND CONSONANTS
==========

1. Start the program.
2. Declare integer counters vow_count and cons_count, and initialize both to zero.
3. Define patterns for vowels: a, e, i, o, u and A, E, I, O, U. Increment vow_count when a vowel is encountered.
4. Define a pattern for alphabetic characters other than vowels as consonants. Increment cons_count when a consonant is encountered.

5. Ignore other characters, including spaces, digits and special characters.
6. Accept the input string and pass it to the Lex analyzer using yylex().
7. Scan the input character by character and match each character against the specified patterns.
8. Whenever a consonant is matched, increment the consonant counter.
9. Whenever a vowel is matched, increment the vowel counter.
10. After the complete input has been scanned, display the total number of vowels.
11. Display the total number of consonants.
12. Stop the program.

==========
EXPERIMENT 6 – RECOGNIZE A VALID ARITHMETIC EXPRESSION USING YACC
==========

1. Start the program.
2. Define the tokens required for arithmetic expressions, such as NUMBER and ID.
3. Define the arithmetic operators: +, −, ∗ and /.
4. Define operator precedence and associativity using YACC declarations.
5. Define grammar rules for valid arithmetic expressions containing numbers, identifiers, operators and parentheses.
6. In the Lex file, define a pattern for numbers. When a number is found, convert it to an integer and return the NUMBER token to YACC.
7. Define a pattern for identifiers and return the appropriate token.
8. For valid arithmetic symbols such as +, −, ∗, /, (and), return the corresponding character/token to YACC.
9. Read an arithmetic expression entered by the user.
10. Let Lex scan the input and generate tokens.
11. Let YACC receive the tokens and check them against the grammar rules.
12. If all tokens satisfy the grammar, accept the input as a valid arithmetic expression.
13. If the expression does not satisfy the grammar, call yyerror() and display an invalid-expression message.
14. If valid, calculate and display the result according to the grammar's semantic actions.
15. Stop the program.

==========
EXPERIMENT 7 – RECOGNIZE A VALID IDENTIFIER USING YACC
==========

1. Start the program.
2. Define the tokens letter and digit in the YACC file.
3. Define the grammar for a valid identifier: the first character

must be a letter; subsequent characters may be letters, digits or underscores.
4. In the Lex file, define a pattern for letters and return the letter token when a letter is encountered.
5. Define a pattern for digits and return the digit token when a digit is encountered.
6. Define a pattern for underscore and return it as a character token.
7. Ignore the newline or use it to terminate the complete identifier input.
8. Accept an identifier from the user.
9. Let Lex scan the identifier character by character and generate the corresponding tokens.
10. Let YACC receive the tokens.
11. Check that the first token is a letter.
12. If the first character is a letter, check that all remaining characters follow the identifier grammar.
13. If the complete string satisfies the grammar, mark the identifier as valid.
14. If the first character is a digit, or any remaining character is invalid, reject the identifier.
15. Display whether the identifier is valid or invalid.
16. Stop the program.

==========
EXPERIMENT 8 – IMPLEMENT A CALCULATOR USING LEX AND YACC
==========

1. Start the program.
2. Define a NUMBER token in the YACC file for integer values.
3. Define the arithmetic operators +, −, ∗, / and %.
4. Define parentheses for grouping expressions.
5. Specify operator precedence and associativity using YACC declarations.
6. Define grammar rules for arithmetic expressions containing numbers, addition, subtraction, multiplication, division, modulus and parenthesized expressions.
7. Define semantic actions to perform the arithmetic operations:
   - Addition: add the two operands.
   - Subtraction: subtract the second operand from the first.
   - Multiplication: multiply the operands.
   - Division: divide the first operand by the second.
   - Modulus: find the remainder.
8. In the Lex file, define a pattern to recognize one or more digits.
9. Convert each recognized number to an integer and return the NUMBER token to YACC.
10. Return arithmetic operators and parentheses directly to YACC.
11. Ignore unnecessary spaces and tabs.
12. Read an arithmetic expression entered by the user.
13. Let Lex scan the expression and convert it into tokens.
14. Let YACC receive the tokens and check whether the expression

follows the grammar.
15. If the expression is syntactically valid, execute the semantic actions and evaluate it.
16. Store and display the calculated result.
17. If the expression does not follow the grammar, call yyerror() and display an invalid-expression message.
18. If no syntax error occurs, indicate that the entered arithmetic expression is valid.
19. Stop the program.

==========
EXPERIMENT 9 – FIND THE ε-CLOSURE OF EACH NFA STATE
==========

1. Start.
2. Read the number of alphabet symbols.
3. Read the alphabet symbols, using ε to represent an epsilon transition.
4. Read the number of states in the NFA.
5. Read the number of transitions.
6. For each transition, read its source state, input symbol and destination state.
7. For each state i, initialize its ε-closure with state i itself.
8. Check all transitions originating from state i.
9. If a transition has ε as its input symbol, add its destination state to the ε-closure. Continue checking ε-transitions from each newly added state.
10. Repeat until no new state can be added to the ε-closure.
11. Display the ε-closure of every state.
12. Stop.

==========
EXPERIMENT 14 – SHIFT REDUCE PARSER FOR A GIVEN EXPRESSION
==========

1. Start.
2. Define the operator-precedence table for the operators and symbols.
3. Read the input expression, with $ as the end marker.
4. Push $ onto the stack.
5. Set the input pointer i = 0.
6. Display the current stack and the remaining input.
7. Check whether the stack contains $ and the input contains $.
8. If both contain $, display “Valid Expression” and stop.
9. Otherwise, get the current input symbol. Find its position in the precedence table, and find the position of the top operator in the stack.
10. Obtain the precedence relationship from the table.
11. If the relationship is < or =:
    a. Push the current input symbol onto the stack.

    b. Increment the input pointer.
    c. Display “Shift”.
12. If the relationship is >:
    a. Pop the top symbol from the stack.
    b. Continue popping until the appropriate < relationship is obtained.
    c. Display “Reduce”.
13. If no valid precedence relationship exists, display “Invalid Expression” and stop.
14. Repeat steps 6–13 until the expression is accepted or rejected.
15. Stop.

==========
EXPERIMENT 15 – CONSTANT PROPAGATION
==========

1. Start.
2. Declare integer variables x, y and z.
3. Assign a constant value to x.
4. Assign an expression involving x to y.
5. Since x is known at compile time, replace x with its constant value in the expression for y.
6. Evaluate the expression and replace y with its constant value.
7. Use the constant value of y to evaluate the expression assigned to z.
8. Replace z with its calculated constant value.
9. Display the final value of z.
10. Stop.